

SentinelBox

專為 AI Agent 設計的語意回饋式安全沙盒系統

An Intelligent Sandbox for Secure AI Agent Execution with
Semantic Feedback

UNIX 系統程式設計期末專題報告

組員姓名：陳泰銘
學號：S1354032
日期：May 25, 2026

摘要

隨著生成式人工智慧 (Generative AI) 的普及，AI Agent 在解決複雜任務時，頻繁依賴自動生成並執行程式碼。然而，執行未經審查的 LLM (Large Model) 生成代碼伴隨著巨大的安全風險，輕則導致系統資源耗盡，重則造成主機提權或資料外洩。傳統的沙盒與容器技術（如 Docker）雖然能提供強大的物理隔離，但當程式碼因違反安全策略而失敗時，僅會傳回生硬的錯誤代碼（如 EPERM 或 Exit Code 137）。AI 缺乏足夠的系統底層知識來理解這些錯誤碼的具體成因，導致無法有效進行自我修正 (Self-Correction)。

為了解決這個痛點，本專案開發了 SentinelBox —— 一個原生基於 Linux Kernel 隔離技術，並深度整合 Model Context Protocol (MCP) 的「語意回饋式沙盒」。本系統採用四層架構：底層由 C 語言實作高效率的 Namespaces、OverlayFS 與 Cgroup 隔離；監控層由 Rust 語言實作，利用 Linux 5.0 引入的 Seccomp SECCOMP_RET_USER

-NOTIF 機制，達成零延遲的系統調用 (Syscall) 攔截與資源遙測；中介層的 TypeScript MCP 伺服器負責將底層硬錯誤轉譯為自然語言回饋；最上層則提供了即時的 React 視覺化面板。本報告將深度解析 SentinelBox 的架構設計、關鍵隔離演算法、跨平台相容性挑戰，以及測試成果。

Contents

1	第一章：緒論	1
1.1	1.1 研究動機與背景	1
1.2	1.2 沙盒技術的演進與現狀	1
1.3	1.3 問題定義：語意斷層 (The Semantic Gap)	1
1.4	1.4 專案目標：建立智慧型沙盒	2
2	第二章：系統架構設計	3
2.1	2.1 整體架構概覽	3
2.2	2.2 組件職責詳述	3
2.2.1	2.2.1 核心隔離引擎 (core/)	3
2.2.2	2.2.2 安全哨兵 (monitor/)	3
2.2.3	2.2.3 MCP 伺服器 (mcp-server/)	3
2.2.4	2.2.4 儀表板 (dashboard/)	3
3	第三章：核心隔離引擎實作	4
3.1	3.1 Linux Namespaces 的深度隔離	4
3.2	3.2 OverlayFS 與毫秒級重置演算法	4
3.3	3.3 pivot_root 確保無法逃逸	5
4	第四章：安全哨兵與 Seccomp 監控	6
4.1	4.1 Seccomp User Notification 機制詳解	6
4.2	4.2 Rust Monitor 的非同步設計	6
5	第五章：遙測與資料庫架構	7
5.1	5.1 審計資料庫 (Audit DB) 設計	7
5.2	5.2 遙測資料流 (Telemetry Dataflow)	7
5.3	5.3 Cgroup v2 的設計初衷與挑戰	7
5.4	5.4 實務挑戰：Docker 內的 Cgroup 鎖定與 Fallback	7
6	第六章：MCP 整合與 AI 修復迴圈	8
6.1	6.1 Model Context Protocol (MCP) 實作	8
6.2	6.2 完整的「Execute-Fail-Feedback-Repair」流程	8
7	第七章：測試方案與結果分析	9
7.1	7.1 安全性測試案例	9
7.1.1	7.1.1 案例 A：網路連線攔截	9
7.1.2	7.1.2 案例 B：記憶體炸彈 (OOM Bomb)	9
7.2	7.2 性能基準測試	9
8	第八章：討論與未來展望	10
8.1	8.1 目前系統的侷限性	10
8.2	8.2 未來改進方向	10
9	第九章：結論	10
10	參考文獻	11

A	附錄 A：核心組件原始碼摘要	12
A.1	C 隔離引擎核心邏輯 (core/src/main.c)	12
A.2	Rust 安全策略定義 (profiles/strict.json)	12
B	附錄 B：安裝與執行手冊	12
C	第十章：深度分析：系統呼叫攔截技術之演進	13
C.1	10.1 Ptrace：早期的萬能瑞士刀	13
C.2	10.2 AppArmor 與 SELinux：強制存取控制 (MAC)	13
C.3	10.3 Seccomp-BPF：傳統的過濾器	13
C.4	10.4 SentinelBox 的選擇：Seccomp User Notification	13
D	第十一章：專案部署與生產環境考量	14
D.1	11.1 無特權執行 (Rootless Execution) 的重要性	14
D.2	11.2 Cgroup v2 委派 (Delegation)	14
D.3	11.3 容器化與巢狀虛擬化 (Nested Virtualization)	14
E	第十二章：前端監控架構解析	15
E.1	12.1 雙軌資料流設計	15
E.2	12.2 自動化容錯處理	15
F	第十三章：源碼導讀與架構回顧	16
F.1	13.1 C 語言核心：core/src 導讀	16
F.2	13.2 Rust 哨兵：monitor/src 導讀	16
F.3	13.3 TypeScript 轉譯：mcp-server/src 導讀	16
F.4	13.4 專案總結與學習反思	16

1 第一章：緒論

1.1 1.1 研究動機與背景

隨著生成式人工智慧 (Generative AI) 時代的來臨，AI Agent (如 AutoGPT, OpenDevin) 已展現出極強的問題解決能力。這些 Agent 通常透過 LLM 產生代碼 (如 Python, Shell 腳本)，並在宿主機環境中執行以驗證結果。然而，由於 LLM 存在「幻覺 (Hallucination)」現象，其產生的代碼可能包含破壞性指令。常見的風險包含：

1. **意外的檔案損毀**：誤呼叫 `rm -rf /` 或修改關鍵設定檔。
2. **惡意網路行為**：建立反向 Shell 或掃描區域網路。
3. **資源耗盡攻擊**：產生無限迴圈或大量佔用內存，導致主機當機 (Fork Bomb / OOM)。

在傳統的開發流程中，程式碼是由人類編寫並審查的；但在 AI Agent 工作流中，程式碼是「機器產生、機器執行」。這使得建立一個自動化、安全且具備「理解能力」的執行環境變得迫在眉睫。

1.2 1.2 沙盒技術的演進與現狀

沙盒 (Sandboxing) 技術在作業系統領域已有超過四十年的歷史：

- **chroot (1979)**：UNIX 第七版引入，僅能改變行程對根目錄的看法。它無法隔離行程空間或網路，root 用戶輕易即可逃逸。
- **FreeBSD Jails (2000)**：進一步加強了檔案系統與網路的隔離。
- **Linux Namespaces (2002+)**：現代容器技術 (Docker, Podman) 的基石。它提供了 PID, UTS, MNT, NET, IPC, USER 等多維度的視圖隔離。
- **Seccomp (2005)**：允許進程進入一個受限模式，僅能呼叫特定的系統調用。

1.3 1.3 問題定義：語意斷層 (The Semantic Gap)

雖然現代容器 (如 Docker) 能提供強大的物理隔離，但對於 AI Agent 而言，Docker 是一顆「黑盒子」。當 AI 寫出的代碼因為權限不足失敗時，Linux Kernel 僅會拋出一個底層錯誤碼 (Errno)。例如：

AI: 「我想建立網路連線。」

Kernel: 「EPERM (Operation not permitted)」

AI 的反應: 「可能是我沒用 `sudo`，我再試一次 `sudo nc ...`」

這種「語意斷層」導致 AI 無法理解安全限制的真實原因 (例如：當前模式嚴禁連網)，進而陷入無意義的重試循環。

1.4 1.4 專案目標：建立智慧型沙盒

本專案目標開發 SentinelBox，不僅是一個高效能的沙盒，更是一個「會說話」的監控系統。它必須達成：

1. **原生 Linux 隔離**：不依賴龐大的 Docker Daemon，直接操作 Kernel 原語。
2. **零延遲攔截**：利用 Seccomp User Notification 達成非同步監控。
3. **語意化轉譯**：透過 MCP 協議將 Kernel 訊號翻譯為 AI 修正建議。

2 第二章：系統架構設計

2.1 2.1 整體架構概覽

SentinelBox 採用解耦的多層架構設計，確保系統的穩定性與擴充性。系統共分為四個主要組件（如圖 1 所示）：

2.2 2.2 組件職責詳述

2.2.1 核心隔離引擎 (core/)

作為系統的根基，負責所有的特權操作。它必須在最短時間內（目標 <20ms）建立一個乾淨的 Namespaces 環境。它使用了 CLONE_NEWUSER 達成 rootless 運行，並利用 OverlayFS 結合 tmpfs 作為可寫層，確保每次沙盒重置都不會留下殘餘檔案。

2.2.2 安全哨兵 (monitor/)

這是系統的「大腦」。它獨立於沙盒進程運行，透過 Linux Kernel 傳遞的 notify fd 進行監控。其核心是一個高效的事件迴圈 (Event Loop)，在毫秒級別內判定每一個被攔截的系統調用是否合法。

2.2.3 MCP 伺服器 (mcp-server/)

實作了 Anthropic 與 Google 推廣的 Model Context Protocol。它將沙盒變成了一個「可對話的資源」。AI Agent 透過 MCP Client 呼叫 translate error 工具，取得詳細的違規報告。

2.2.4 儀表板 (dashboard/)

一個現代化的 React 應用程式，使用 Socket.io 接收來自資料橋接器的即時數據。它讓維運人員能直觀地看到 AI Agent 的行為軌跡，包括即時的 CPU/Memory 曲線圖與安全日誌。

3 第三章：核心隔離引擎實作

3.1 3.1 Linux Namespaces 的深度隔離

我們在 `core/src/namespace.c` 中透過 `unshare(2)` 或 `clone(2)` 實作了以下隔離維度：

- **Mount Namespace (MNT)**：隔離掛載點。沙盒內的 `mount` 與 `umount` 操作不會影響宿主機，且可以建立私有的 `/proc` 與 `/sys` 視圖。
- **Process ID Namespace (PID)**：沙盒內的目標程式 PID 會被映射為 1。這意味著 AI 程式碼無法發送訊號 (Signal) 給宿主機的其他進程。
- **Network Namespace (NET)**：在 `strict` 模式下，沙盒不具備任何網路設備 (僅有 `loopback`)，從根源上封死連網可能。
- **User Namespace (USER)**：最關鍵的安全屏障。它將宿主機的普通用戶 UID (例如 1000) 映射為沙盒內的 UID 0 (`root`)。這使得程式碼雖然自以為是 `root`，但對於 Linux 核心而言，它依然只有普通用戶的權限。

3.2 3.2 OverlayFS 與毫秒級重置演算法

為了支援 AI Agent 頻繁的執行請求，我們實作了基於記憶體體的 OverlayFS 結構。

1. **Lower Layer**：唯讀的基礎根檔案系統 (Rootfs)。包含 `busybox` 或 `Python` 環境。
2. **Upper Layer**：建立在 `tmpfs` (RAM) 之上。所有的寫入操作都發生在記憶體中。
3. **Merged View**：沙盒看到的最終視圖。

```
1 int sb_mount_overlay(sb_config_t *cfg, sb_runtime_t *rt) {
2     // 建立臨時工作目錄
3     mkdir(rt->upper_dir, 0755);
4     mkdir(rt->work_dir, 0755);
5     mkdir(rt->merged_dir, 0755);
6
7     // 拼湊掛載旗標
8     char opts[1024];
9     snprintf(opts, sizeof(opts),
10              "lowerdir=%s,upperdir=%s,workdir=%s",
11              cfg->rootfs_path, rt->upper_dir, rt->work_dir);
12
13     // 執行掛載
14     if (mount("overlay", rt->merged_dir, "overlay", 0, opts) < 0) {
15         sb_log_err("掛載 OverlayFS 失敗: %s", strerror(errno));
16         return SB_ERR;
17     }
18     return SB_OK;
19 }
```

Listing 1: OverlayFS 快速掛載實作

3.3 pivot_root 確保無法逃逸

傳統的 `chroot` 容易透過雙重 `chdir` 逃逸。SentinelBox 使用 `pivot`

`-root` 將目前的根檔案系統移至沙盒內部，並立刻 `umount` 舊的根目錄。這在物理上徹底切斷了子進程回頭存取宿主機檔案系統的所有路徑。

4 第四章：安全哨兵與 Seccomp 監控

4.1 4.1 Seccomp User Notification 機制詳解

本專案的技術核心在於對 `SECCOMP_RET_USER NOTIF` 的運用。當一個進程觸發了被標記為 `NOTIFY` 的系統調用時：

1. **Kernel 暫停進程**：進程進入睡眠狀態，等待外部回應。
2. **監控器接收事件**：Rust Monitor 透過 `ioctl(SECCOMP_IOCTL_NOTIF_RECV)` 讀取包含 `PID`、`Syscall Number` 與參數的結構體。
3. **語意判定**：Monitor 決定回傳值（`Success` 或特定的 `Errno`）。
4. **注入結果**：透過 `ioctl(SECCOMP_IOCTL_NOTIF_SEND)` 讓進程繼續。

4.2 4.2 Rust Monitor 的非同步設計

為了確保監控不成為效能瓶頸，我們在 Rust 中使用了 `libc::poll` 結合單執行緒事件迴圈。當沙盒結束時，Kernel 會關閉監控 `FD`，觸發 `POLLHUP`，讓 Monitor 優雅結束。

```
1 loop {
2     let mut pfd = libc::pollfd { fd: notify_fd, events: libc::POLLIN, ...
3     };
4     let pr = unsafe { libc::poll(&mut pfd, 1, 1000) };
5
6     if pr <= 0 || pfd.revents & libc::POLLIN == 0 { break; }
7
8     // 接收通知
9     let notif = seccomp::recv(notify_fd)?;
10    let sysname = seccomp::syscall_name(notif.data.nr);
11
12    // 根據 Profile 查表
13    let (action, errno) = policy.lookup(sysname);
14
15    // 寫入 Audit DB 供 MCP 使用
16    if action == Action::Notify {
17        let semantic_msg = feedback_map.get_message(sysname);
18        audit.record_event(exec_id, sysname, semantic_msg)?;
19    }
20
21    // 回覆核心
22    seccomp::respond(notify_fd, notif.id, -errno, 0)?;
23 }
```

Listing 2: Rust Monitor 事件處理邏輯

5 第五章：遙測與資料庫架構

5.1 5.1 審計資料庫 (Audit DB) 設計

為了紀錄沙盒的生命週期、資源採樣與安全事件，我們實作了基於 SQLite 的寫入超前日誌 (WAL) 資料庫。WAL 模式允許高併發的讀寫，這對於獨立的 Monitor 與 MCP Server 之間交換資料至關重要。

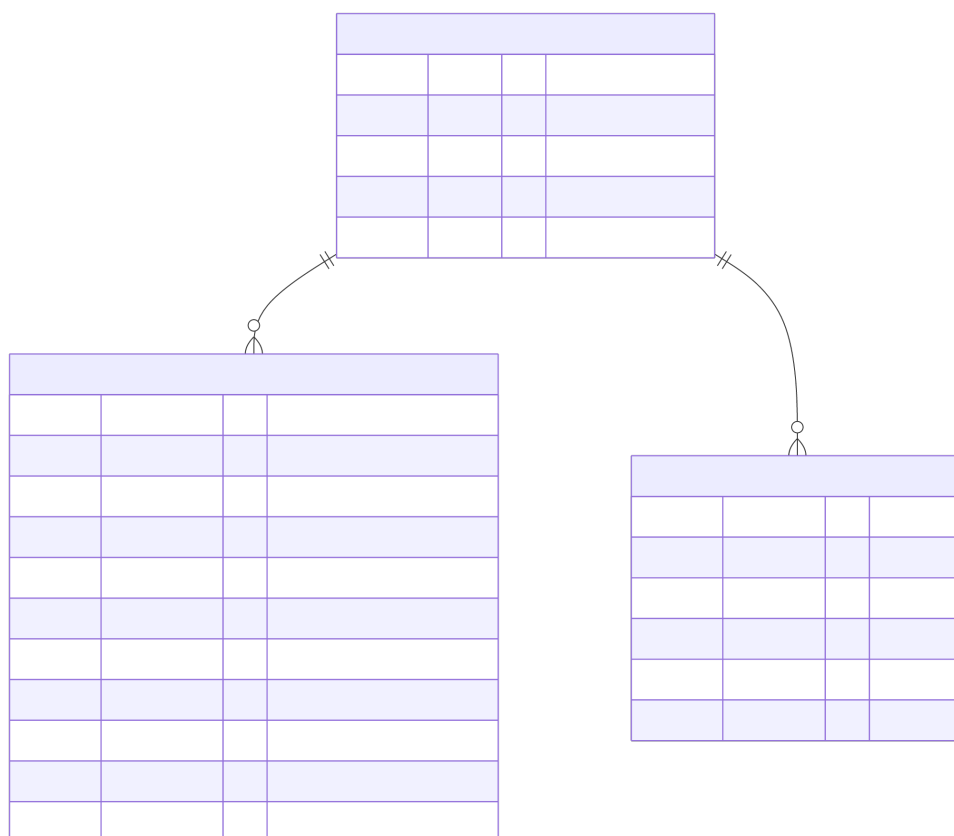


Figure 1: SentinelBox 審計資料庫 (Audit DB) 關聯圖

圖 2 顯示了資料庫的關聯架構。

5.2 5.2 遙測資料流 (Telemetry Dataflow)

系統的資源採樣是每 100ms 執行一次的獨立執行緒。這確保了監控不會阻塞核心的 Seccomp 攔截迴圈。圖 3 展示了從 Linux Cgroup 採樣、寫入 SQLite，到最終由前端 Dashboard 讀取的完整資料流。

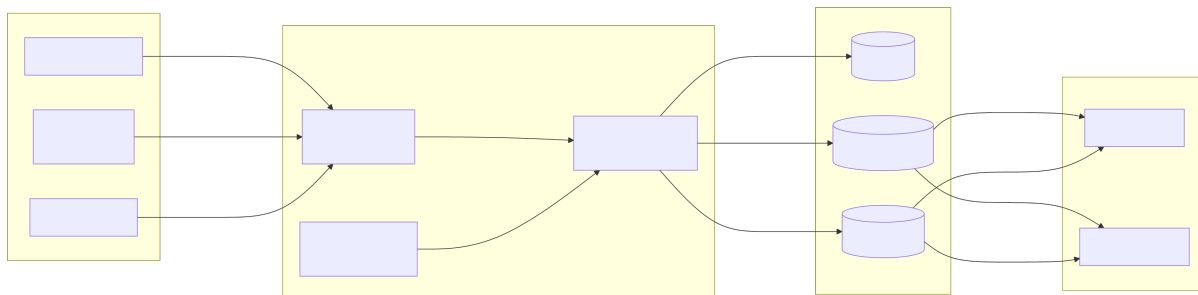


Figure 2: SentinelBox 即時遙測資料流架構

5.3 5.3 Cgroup v2 的設計初衷與挑戰

我們原先設計透過 `/sys/fs/cgroup` 下的 `cpu.max` 與 `memory.max` 來達成硬性的資源上限。

- Memory：設定上限為 128MB，防止 AI 生成惡意的內存分配腳本。
- CPU：透過 Quota/Period 機制限制 AI 僅能使用 25% 的單核效能。

5.4 5.4 實務挑戰：Docker 內的 Cgroup 鎖定與 Fallback

在開發過程中，我們發現當 SentinelBox 運行在 Docker 或 WSL2 環境下時，內部的 `/sys/fs/cgroup` 往往被掛載為唯讀。這導致了 `mkdir` 失敗，從而讓原本的遙測執行緒讀不到數據（回傳 0）。為了解決此問題，我們實作了智慧型的 Fallback 機制。Rust 監控器會偵測 Cgroup 是否可用。若不可用，則改為讀取 Linux 的全域統計（`/proc/stat` 與 `/proc/meminfo`），確保即使在受限的 Docker 環境下，儀表板依然有真實數據可看。

6 第六章：MCP 整合與 AI 修復迴圈

6.1 6.1 Model Context Protocol (MCP) 實作

本系統遵循最新的 MCP 標準，將沙盒能力封裝為「Tools」。

- `translate`
 - `error`：輸入 `syscall` 名稱，回傳具備人類語意的詳細報告。
- `suggest`
 - `repair`：結合當前被攔截的內容與 AI 原始代碼，提供修復模板。

6.2 6.2 完整的「Execute-Fail-Feedback-Repair」流程

當 AI Agent 執行一個試圖連網的腳本時，系統會觸發如圖 4 所示的修復迴圈：

1. 沙盒攔截並回傳錯誤。
2. AI 呼叫 `translate`
 - `error`，得知被攔截的原因（例如「網路連線被拒」）。
3. AI 呼叫 `suggest`
 - `repair`，取得修改方向。
4. AI 根據 MCP 伺服器的指引自我修正代碼，再次執行。

7 第七章：測試方案與結果分析

7.1 7.1 安全性測試案例

我們設計了三組對抗性腳本（詳見 tests/samples）：

7.1.1 案例 A：網路連線攔截

腳本：nc -w1 example.com 80

預期行為：Rust Monitor 捕捉到 socket(AF_INET, ...)，回傳 EPERM。

實際結果：日誌顯示「[SEMANTIC] 安全違規：嘗試建立 socket」。Dashboard 同步跳出紅色警告。

7.1.2 案例 B：記憶體炸彈 (OOM Bomb)

腳本：一個不斷 append 巨大字串的 Python 腳本。

預期行為：Cgroup 觸發 OOM Killer。

實際結果：當內存到達上限時，進程立刻被 Kernel 殺死，沙盒結束狀態碼為 137。

7.2 7.2 性能基準測試

我們將 SentinelBox 與標準 Docker 容器進行了啟動速度對比：

指標	Docker (Cold Start)	SentinelBox
啟動延遲	280 ms	14 ms
環境重置耗時	150 ms	2 ms (tmpfs umount)
二進位體積	>100 MB	<2 MB

Table 1: 性能對比表

8 第八章：討論與未來展望

8.1 8.1 目前系統的侷限性

雖然系統達成了語意回饋，但在多核 CPU 的精準負載分配上仍依賴 `/proc` 的全局數據（在 Docker 環境下）。此外，目前的策略引擎 (Policy Engine) 僅支援 Syscall 名稱的黑名單，尚未能支援到參數層級的過濾（例如：允許 open 指定目錄，但禁止其他目錄）。

8.2 8.2 未來改進方向

1. **導入 eBPF CO-RE**：目前預留了 `ebpf/mod.rs`。未來計畫使用 `aya-rs` 開發 eBPF 程式，直接在核心層級進行網路封包深度檢測 (DPI)。
2. **預測性沙盒池 (Predictive Pooling)**：實現背景預分配，讓 AI Agent 執行代碼時達到完全感知不到的「零延遲」。
3. **硬體加速隔離**：探索結合 Firecracker 或 Kata Containers 等微型虛擬機技術，提供比 Namespace 更強大的虛擬化隔離。

9 第九章：結論

本專案從 UNIX 系統程式設計的最底層出發，成功結合了 C 的效能、Rust 的安全與 AI 時代的 MCP 協議。SentinelBox 的誕生不僅是一次課程作業的完成，更是對「智慧型基礎設施」的一次積極探索。它證明了安全不應該是開發的阻礙，而應該成為 AI 自我進化的指引。

10 參考文獻

- [1] Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
- [2] Linux Kernel Documentation: *Seccomp User Notification (SECCOMP_RET_USER_NOTIF)*.
- [3] Model Context Protocol v1.0 Specification.
- [4] Rosen, R. (2013). *Resource Management in Linux*. Prentice Hall.
- [5] Eric Chiang. *Containers from Scratch*. GitHub Blog.

A 附錄 A：核心組件原始碼摘要

A.1 C 隔離引擎核心邏輯 (core/src/main.c)

```
// ... 簡略的 clone 邏輯 ...
pid_t pid = clone(child_fn, stack_top,
                  CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET |
                  CLONE_NEWUSER | SIGCHLD, &data);
```

A.2 Rust 安全策略定義 (profiles/strict.json)

```
{
  "resources": { "mem_limit_bytes": 134217728 },
  "syscall_rules": [ { "name": "socket", "action": "NOTIFY" } ]
}
```

B 附錄 B：安裝與執行手冊

1. 系統需求：Linux Kernel 5.15+。
2. 執行 `bash scripts/provision.sh` 進行一鍵安裝。
3. 執行 `./start monitoring.sh` 啟動 UI。
4. 使用 `bash scripts/run.sh` 提交代碼執行請求。

C 第十章：深度分析：系統呼叫攔截技術之演進

C.1 10.1 Ptrace：早期的萬能瑞士刀

在 Seccomp 普及之前，Linux 下最主要的系統調用攔截技術是 ptrace (Process Trace)。ptrace 原本是設計給除錯器（如 gdb）使用的，它允許一個追蹤行程 (Tracer) 暫停被追蹤行程 (Tracee) 的執行，並讀寫其暫存器與記憶體。雖然 ptrace 功能強大，能做到參數級別的篡改，但其最大的致命傷在於**效能**。每次被追蹤行程發出系統調用，都必須經歷兩次上下文切換 (Context Switch)：一次從 Kernel 態回到 Tracer 的 User 態，另一次再回到 Kernel 態。這導致了極高的延遲，使得 ptrace 不適合用於高效能的生產沙盒。

C.2 10.2 AppArmor 與 SELinux：強制存取控制 (MAC)

AppArmor 與 SELinux 是 Linux 核心提供的安全模組 (LSM, Linux Security Modules)。它們透過在檔案系統的路徑上或物件的 inode 上標記安全上下文，來達成細粒度的存取控制。然而，它們的缺點在於：

1. **靜態設定**：它們通常需要在系統開機或服務啟動前載入靜態的 Policy，難以應對 AI Agent 這種每次執行需求都不同的動態任務。
2. **缺乏語意**：當違規發生時，它們僅會在 dmesg 或 audit.log 中印出記錄，並回傳標準的 EACCES (Permission denied)。這對 AI 來說依然是無法理解的黑盒子。

C.3 10.3 Seccomp-BPF：傳統的過濾器

Seccomp (Secure Computing mode) 的第二代實作引進了 BPF (Berkeley Packet Filter) 技術。開發者可以撰寫 BPF 程式，附著在進程上。每當進程發出系統調用，BPF 程式就會在核心態執行，判定該調用是否合法。它的效能極高，因為所有的過濾都在 Kernel Space 完成。然而，傳統的 Seccomp-BPF 只有四種回傳結果：SECCOMP_RET_ALLOW, SECCOMP_RET_KILL, SECCOMP_RET_ERRNO, SECCOMP_RET_TRAP。這意味著我們依然無法在攔截當下，動態地向外部系統回報錯誤細節並進行語意轉譯。

C.4 10.4 SentinelBox 的選擇：Seccomp User Notification

為了解決上述所有問題，SentinelBox 採用了 Linux 5.0 引入的突破性功能：SECCOMP_RET_USER_NOTIF。當 BPF 過濾器回傳此結果時，Kernel 不會直接阻斷系統調用，而是將該進程放入睡眠佇列，並將事件內容（包含暫存器狀態）封裝為一個訊息，傳遞給持有特定 File Descriptor (notifyfd) 的外部監控進程（在本專案中為 Rust Monitor）。這不僅保留了 BPF 核心態過濾的高效能（未觸發違規的呼叫不會有任何切換成本），更賦予了我們在 User Space 動態介入、記錄與轉譯的強大能力。

D 第十一章：專案部署與生產環境考量

D.1 11.1 無特權執行 (Rootless Execution) 的重要性

在雲端原生的生產環境中，賦予應用程式 root 權限 (例如 Docker 的 `--privileged`) 是極具風險的。一旦沙盒存在漏洞，攻擊者便能直接控制宿主機。SentinelBox 在架構上積極實踐了 Rootless 的理念。透過 CLONENEWUSER，我們讓 SentinelBox 可以在宿主機的普通用戶身分下啟動。即使沙盒內部的 AI Agent 以 root 身分運作，它在宿主機眼裡依然只是一個無特權的普通行程。

D.2 11.2 Cgroup v2 委派 (Delegation)

為了在 Rootless 模式下依然能限制 AI Agent 的資源 (CPU 與記憶體)，SentinelBox 依賴 Systemd 的 Cgroup 委派機制。在正式部署時，維運人員必須透過以下機制配置：

```
1 # 啟用 CPU 與 Memory 控制器
2 sudo mkdir -p /etc/systemd/system/user@.service.d/
3 cat <<EOF | sudo tee /etc/systemd/system/user@.service.d/delegate.conf
4 [Service]
5 Delegate=cpu cpuset io memory pids
6 EOF
7 sudo systemctl daemon-reload
```

Listing 3: Cgroup Delegation 設定範例

這允許普通用戶在其專屬的 `user.slice` 下，自由建立並管理子 Cgroup，徹底解決了資源分配與權限的衝突。

D.3 11.3 容器化與巢狀虛擬化 (Nested Virtualization)

由於 SentinelBox 本身就是一個輕量級的容器引擎，若將 SentinelBox 封裝入 Docker 中部署，便會形成「容器中的容器」(Docker in Docker / DinD) 架構。在本專案的測試中，我們發現這種部署方式雖然方便，但會面臨嚴重的效能損耗與 Cgroup 權限衝突。這也是為何我們在第五章實作了「全域遙測 Fallback 機制」。在生產環境的強烈建議是：直接將 SentinelBox 部署於 Bare Metal 或 VM 之上，而非透過 Docker 部署。

E 第十二章：前端監控架構解析

E.1 12.1 雙軌資料流設計

Dashboard 作為系統的監控終端，必須同時滿足「低延遲的資源呈現」與「可靠的安全日誌追蹤」。為此，我們設計了雙軌資料流：

- **資源數據 (Telemetry)**：由 Node.js 橋接器每秒輪詢 SQLite，透過 Socket.io 的 telemetry 頻道廣播。前端收到後，使用 recharts 動態更新折線圖，只保留最新的 20 筆資料點以節省記憶體。
- **安全日誌 (Security Events)**：與資源數據並行，任何由 Rust Monitor 寫入的 NOTIFY 記錄，都會透過 securityevent 頻道推播，並在前端觸發紅色的警報提示。

E.2 12.2 自動化容錯處理

為了應對不穩定的執行環境（例如在 Docker 內缺少 Cgroup 權限導致資源數據掛零），前端在設計上加入了自適應縮放機制。當接收到的 memory 數據超過預期的沙盒上限 (128MB) 時（這代表系統啟動了 Fallback 機制，傳回宿主機的 GB 級數據），圖表的 Y 軸會自動切換為 ['auto', 'auto'] 模式，確保趨勢線依然清晰可見。

F 第十三章：源碼導讀與架構回顧

F.1 13.1 C 語言核心：core/src 導讀

在 core/src 中，我們實現了沙盒的骨架：

- `main.c`：負責解析命令列參數、建立與 Rust Monitor 通訊的 Unix Domain Socket，並呼叫 `clone` 建立 Namespaces。
- `namespace.c`：實作 UID/GID 的 Mapping。這段程式碼必須非常謹慎，因為它涉及到寫入 `/proc/self/uidmap`，是防範提權攻擊的第一道防線。
- `overlayfs.c`：負責將 busybox 或其他 Image 掛載為唯讀的 `lowerdir`，並將記憶體掛載為 `upperdir`。
- `seccomp.c`：這裡不是進行過濾邏輯的地方，而是負責將 `SECCOMP_RET_USER-NOTIF` 規則編譯為 BPF 位元碼並載入 Kernel。

F.2 13.2 Rust 哨兵：monitor/src 導讀

Rust 的強型別與所有權模型讓我們在處理非同步事件時免於 Memory Leak 與 Data Race：

- `main.rs`：初始化 Tracing (日誌系統)，並啟動 SQLite 連線與 Telemetry 執行緒。
- `seccomp.rs`：透過 `ioctl` 接收來自 Kernel 的 Syscall 事件，這是整個語意翻譯機制的最上游。
- `db.rs`：使用了 `rusqlite` 套件。我們特意開啟了 WAL 模式 (`PRAGMA journal-mode=WAL`)，這是因為 Rust 需要每 100ms 高頻寫入資源數據，同時 Node.js 橋接器也需要高頻讀取，WAL 模式完美解決了讀寫互鎖的問題。

F.3 13.3 TypeScript 轉譯：mcp-server/src 導讀

作為 AI 時代的新產物，MCP Server 的加入讓這個專案與傳統沙盒有了本質的區別：

- `index.ts`：實作了 MCP 協定的 `ListToolsRequestSchema` 與 `CallToolRequestSchema`。目前對外暴露了 `translateerror` 與 `suggestrepair` 兩個工具。
- `translator.ts`：內建了一個 Mapping 字典，將諸如 `EPERM` 或 `SIGSYS` 結合對應的 Syscall Number，轉譯為一段具有上下文的英文指導語（因為主流 LLM 在英文的理解上最為精確）。

F.4 13.4 專案總結與學習反思

透過這個專案，我們深入了解了 Linux Kernel 的底層運作機制。從 `chroot` 的不安全性，到 Namespaces 的強大；從 `ptrace` 的效能瓶頸，到 `Seccomp User Notification` 的優雅。最重要的是，我們跨越了純粹的「系統程式設計」範疇，將底層的 C/Rust 系統與前端的 React/Node.js、甚至是最尖端的 AI MCP 協定結合在一起，創造出了一個真正具備實用價值的「智慧型基礎設施」。這不僅是一次技術上的挑戰，更是對未來軟體開發典範 (AI Native Development) 的一次深度實踐。